INF20325

ESTRUCTURAS DE DATOS

Ayudantía 8: Árboles 2-3

Ignacia Nettle Oliveri
inettle@usm.cl

BLOQUE 1

Árboles 2-3

Árboles 2-3

Definición

Un árbol 2-3 es un **árbol ordenado** en el que:

- Cada nodo puede almacenar 1 o 2 elementos.
  - Si el nodo almacena 1 elemento, puede tener 0 o 2 hijos.
  - Si el nodo almacena 2 elementos, puede tener 0 o 3 hijos.
- Todas las hojas se encuentran en el **mismo nivel**.



Búsqueda en Árbol 2-3

Si **nodo no es una hoja**:

Si nodo tiene un solo elemento

- Si `target` es menor al elemento, seguir por hijo izquierdo.
- Si `target` es mayor al elemento, seguir por hijo derecho.

Si nodo tiene dos elementos

Digamos que el nodo tiene los elementos `r1` y `r2`, con $r1 \leq r2$:

- Si `target` es menor que `r1`, seguir por hijo izquierdo.
- Si `target` es mayor que `r1`, pero menor que `r2`, seguir por hijo central.
- Si `target` es mayor a `r2`, seguir por hijo derecho.

Inserción en Árbol 2-3

Al igual que en árboles ABB y AVL, la inserción de nuevos elementos ocurre **en las hojas**.

Sin embargo, el crecimiento del árbol es hacia **arriba**, o hacia la raíz, en lugar de hacia abajo. Esta propiedad es lo que garantiza que las hojas estén siempre en el mismo nivel.

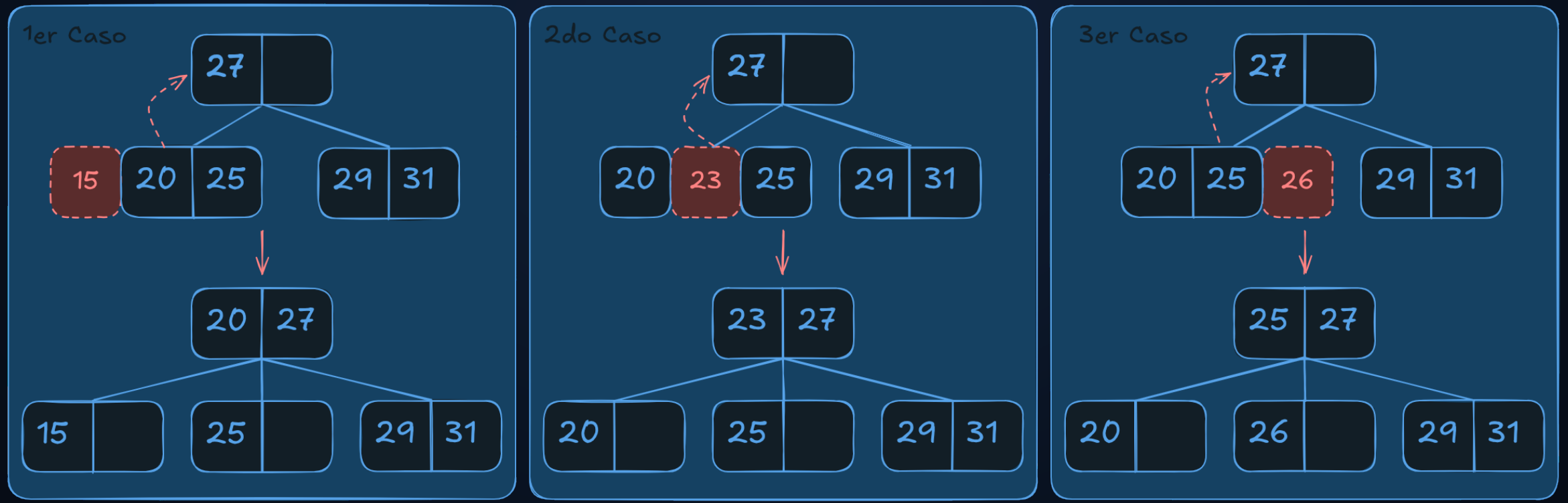
Borrado en árbol 2-3 no se ve en este curso.

Tipos de Inserción

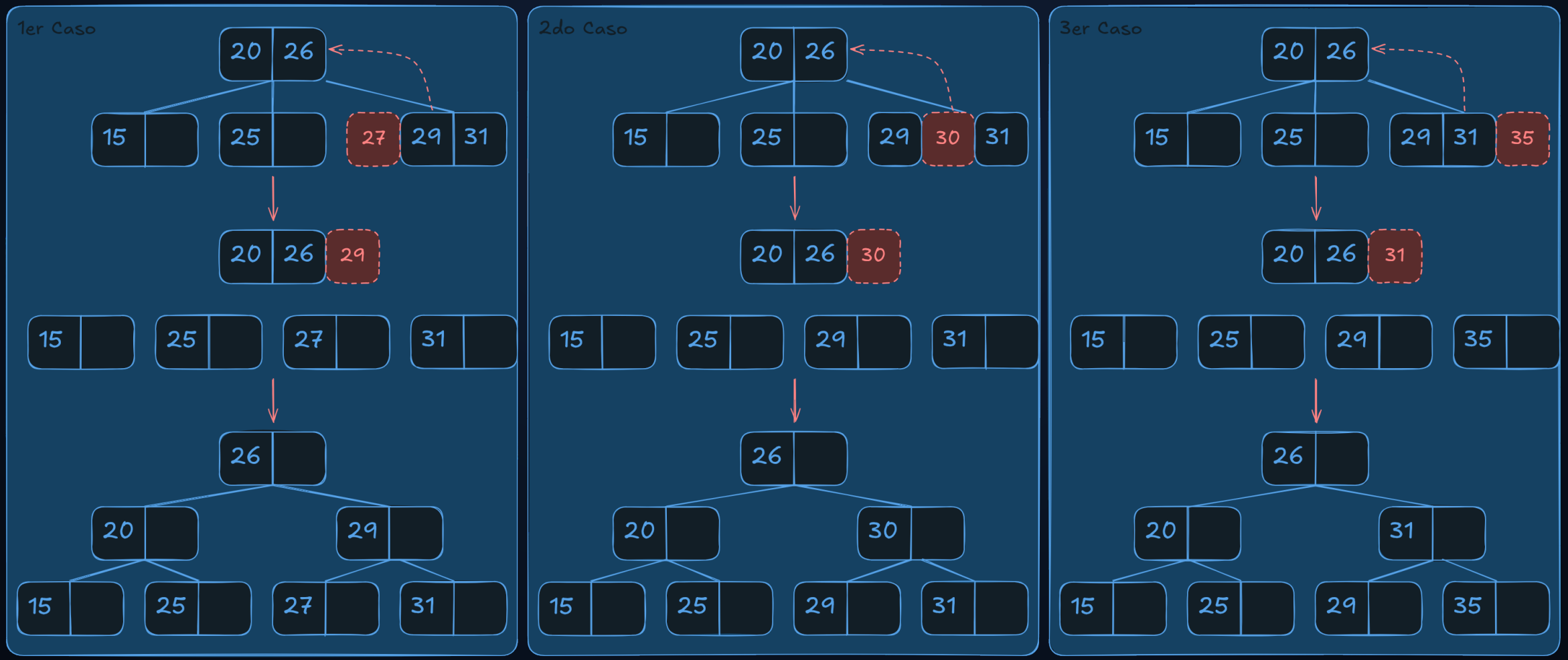
La inserción en Árboles 2-3 tiene dos casos:

- Hay un espacio libre en el nodo para insertar, en cuyo caso se inserta normalmente.
- No hay espacio libre entre los nodos, por lo que el nodo debe **dividirse en dos** y el **valor medio** de los tres elementos se inserta en el padre **recursivamente.**

Tipos de Inserción: `Nodo con espacio libre`



Tipos de Inserción: Nodo sin espacio libre



BLOQUE 2

Ejercicios

Ejercicio 1: Árboles 2-3

Insertar en un árbol 2-3 la siguiente secuencia de números:

$$\{15, 23, 10, 7, 12, 18, 21, 27, 25, 30, 6, 3\}$$

¿Cuál es la **mínima** cantidad de elementos que se deberían insertar al árbol 2-3 resultante para que sea **equivalente a un ABB**?

Ejercicio 2: `Pilas, colas, listas`

Implementar una función que reciba una string con código de *Scheme*, un lenguaje cuyo syntax está basado en paréntesis.

La función debe retornar una **lista** con las **posiciones de inicio y fin de cada set de paréntesis**. En caso de haber error, se retornar una **lista vacía**.

Por ejemplo:

$$(*\ \ (-\ \ 8\ \ 7)\ \ (+\ \ 4\ \ 6)\ \ )$$

Retorna la lista de rangos:

$$(0,\ \ 18),\ \ (3,\ \ 9),\ \ (11,\ \ 17)$$

Considerar que se tiene a disposición un struct `Rango`, que contiene dos enteros: inicio y fin.

CIERRE

¡Eso es todo 🤓!